

Homework 01 R Basics

Due by 11:59pm, Friday, 9.12.25

S&DS 2300/5300/ENV 757

(1) **R Markdown Practice** Change the markdown code below as indicated.

Make this line bold

Make this line italics

Make this line a second level header

- Make this line a bullet point
 - Make this line an indented (or level two) bullet point

LINK (<https://www.nytimes.com/>) (make the word LINK at left link to the New York Times home page AND make it bold)

Make this line look like R Code

Below this line, insert a new R chunk, create a vector called `xvec` that contains the integers 5 through 9, and have R display what is in `xvec`.

```
xvec <- 5:9
xvec
```

```
## [1] 5 6 7 8 9
```

(2) **R Syntax Practice** Modify the R code below to follow good R Syntax practices

```
x <- 5

x <- c(1, 2, 3)

length(x)

for (i in 1:10) {
  x <- 1 + 1
}

x <- 1
y <- c(3, 4)
```

(3) **Data handling**

(3.1) Insert a new R code chunk below.

```
# Empty R chunk
```

(3.2) Read the .csv stored HERE (https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv) into a new data frame and call it "wb". This is similar to the World Bank data I discussed in class two (this is a more current

version). You can get the longer description of each variable HERE (https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024_NMS.csv)

```
wb <- read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv")
```

(3.3) Get the dimension of wb.

```
dim(wb)
```

```
## [1] 217 18
```

(3.4) Get the variable names of wb.

```
names(wb)
```

```
## [1] "Country" "Population" "Rural" "GNI" "Imports"
## [6] "Exports" "Military" "Cell" "Fertility" "Measles"
## [11] "InfMort" "LifeExp" "PM2.5" "CO2" "EnergyUse"
## [16] "Renewable" "Debt" "EU"
```

(3.5) Show the first 5 lines of wb.

```
wb[1:5, ]
```

##	Country	Population	Rural	GNI	Imports	Exports	Military	Cell
## 1	Afghanistan	42239854	73.067	360	37.06956	14.34215	NA	56.55443
## 2	Albania	2745972	35.397	6770	44.70882	31.30916	1.584881	92.31992
## 3	Algeria	45606480	24.732	4490	23.38840	23.88251	4.779438	106.42354
## 4	American Samoa	43914	12.765	NA	92.53333	44.26667	NA	NA
## 5	Andorra	80088	12.226	50080	NA	NA	NA	118.67298
##	Fertility	Measles	InfMort	LifeExp	PM2.5	CO2	EnergyUse	Renewable
## 1	4.523	68	44.8	62.879	46.087094	0.138000720	2.94	20.0
## 2	1.376	86	8.4	76.833	15.707004	1.615083618	2.27	41.9
## 3	2.829	79	18.7	77.129	25.552656	3.943578663	5.61	0.1
## 4	NA	NA	NA	NA	6.715147	0.002258713	NA	0.4
## 5	NA	98	2.6	NA	9.080281	NA	1.89	18.4
##	Debt	EU						
## 1	NA	no						
## 2	56.302323	no						
## 3	3.733707	no						
## 4	NA	no						
## 5	NA	no						

(3.6) Get the data type of each variable.

```
str(wb)
```

```
## 'data.frame':    217 obs. of  18 variables:
## $ Country      : chr  "Afghanistan" "Albania" "Algeria" "American Samoa" ...
## $ Population: num  42239854 2745972 45606480 43914 80088 ...
## $ Rural        : num  73.1 35.4 24.7 12.8 12.2 ...
## $ GNI          : int   360 6770 4490 NA 50080 1870 18710 11590 5960 33410 ...
## $ Imports      : num  37.1 44.7 23.4 92.5 NA ...
## $ Exports      : num  14.3 31.3 23.9 44.3 NA ...
## $ Military     : num  NA 1.58 4.78 NA NA ...
## $ Cell         : num  56.6 92.3 106.4 NA 118.7 ...
## $ Fertility    : num  4.52 1.38 2.83 NA NA ...
## $ Measles      : int   68 86 79 NA 98 37 99 83 95 NA ...
## $ InfMort      : num  44.8 8.4 18.7 NA 2.6 45.7 8.1 8.4 9.2 NA ...
## $ LifeExp      : num  62.9 76.8 77.1 NA NA ...
## $ PM2.5        : num  46.09 15.71 25.55 6.72 9.08 ...
## $ CO2          : num  0.138 1.61508 3.94358 0.00226 NA ...
## $ EnergyUse    : num  2.94 2.27 5.61 NA 1.89 3.1 4.83 3.39 3.91 3.13 ...
## $ Renewable    : num  20 41.9 0.1 0.4 18.4 52.9 0.9 9.2 9.1 8.8 ...
## $ Debt         : num  NA 56.3 3.73 NA NA ...
## $ EU           : chr   "no" "no" "no" "no" ...
```

(3.7) In your code, insert a comment that gives the data type of the variable `GNI` ?

```
# str(wb$GNI)
# This returns: int [1:217] 360 6770 4490 NA 50080 1870 18710 11590 5960 33410 ...
```

(3.8) Create a new object called `wb_Subset` that has only the variables `Country`, `GNI`, `EnergyUse`, and `Measles` (in that order) AND only for countries where `GNI` is greater than 70000. Make sure you show the value of `wb_Subset` .

```
wb_Subset <- subset(wb, GNI > 70000, c(Country, GNI, EnergyUse, Measles))
wb_Subset
```

```
##           Country      GNI EnergyUse Measles
## 22      Bermuda 134640      1.52      NA
## 54      Denmark  73520      1.96      95
## 66  Faroe Islands  74420      NA      NA
## 89      Iceland  73930     12.33      91
## 94      Ireland  79730      1.09      90
## 117  Luxembourg  89200      1.98      99
## 148      Norway  96770      3.43      96
## 160      Qatar  70070      7.20      99
## 189  Switzerland  95490      1.53      96
## 207  United States  76590      4.24      92
```

(3.9) Get summary statistics for `Debt` . Store the results in a new object called `wb_Stats` . Incidentally, `wb_Stats` will be a vector!

```
wb_Stats <- summary(wb$Debt)
wb_Stats
```

```
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.      NA's
##      2.41  32.84   47.35   56.18  66.45  423.64       102
```

(3.10) Get the length of `wb_Stats` .

```
length(wb_Stats)
```

```
## [1] 7
```

(3.11) Get `r` to show the following elements of `wb_Stats` : 1,2,3,5,6 AND round the result to 1 decimal place.

```
round(wb_Stats[1:6], 1)
```

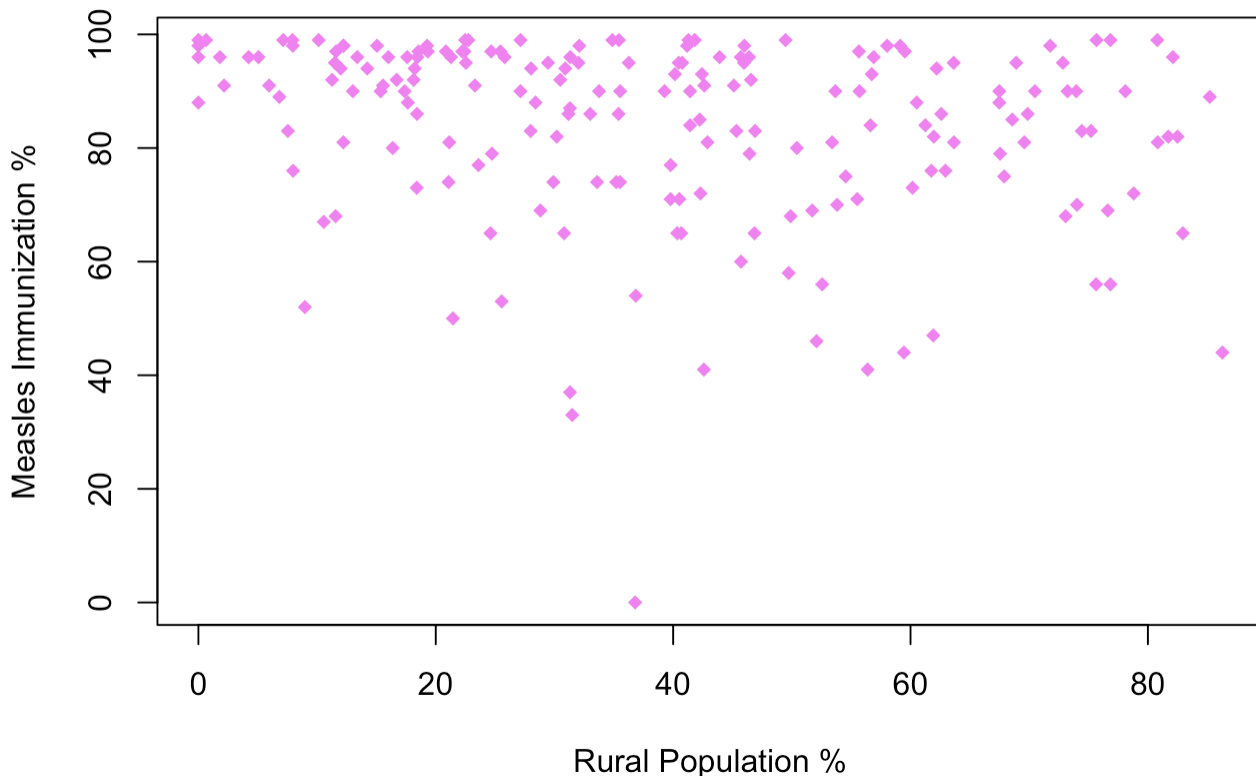
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      2.4   32.8   47.4   56.2   66.4   423.6
```

(4) Plots

(4.1) Using the WB dataset loaded above, make a scatterplot of “Rural” on the x axis and “Measles” on the y axis. Include a main title, axis titles, and a non-default symbol color and symbol type. *Hint: check out ?par or see examples from class 1 or class 3.*

```
plot(wb$Rural,
     wb$Measles,
     col = 'violet',
     pch = 18,
     xlab = "Rural Population %",
     ylab = "Measles Immunization %",
     main = "Measles Immunization as Compared to Rural Population Percent")
```

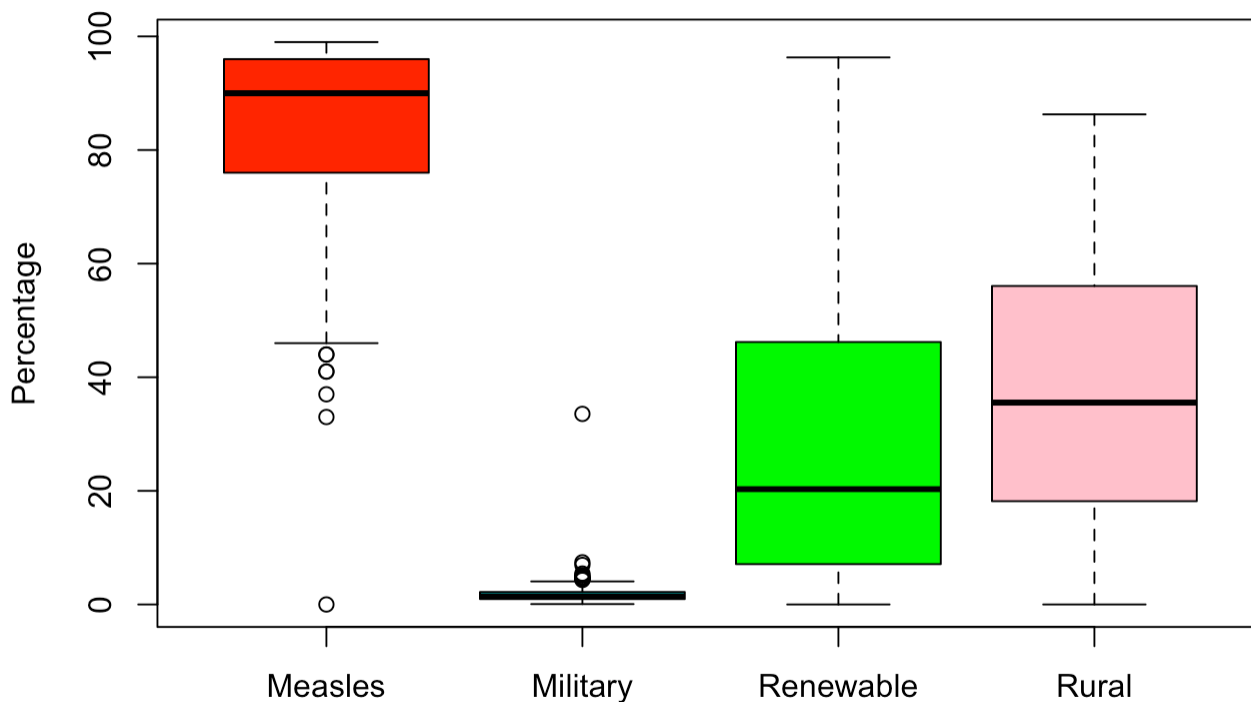
Measles Immunization as Compared to Rural Population Percent



(4.2) Make a boxplot of the the following `wb` variables that are all percentages: `Measles` , `Military` , `Renewable` , `Rural` . You want all four variables on one plot. Ensure the plot has a main title, axis labels, and a unique color for each variable.

```
boxplot(wb$Measles, wb$Military, wb$Renewable, wb$Rural,
        names = c("Measles", "Military", "Renewable", "Rural"),
        col = c('red', 'cyan', 'green', 'pink'),
        ylab = "Percentage",
        main = "Meales, Military, Renewable, and Rural Percentages")
```

Meales, Military, Renewable, and Rural Percentages



(5) Lists The code below creates a list called `aList`

(5.1) Compute the sum of the second element of the list's third element. Store the result into an object named `mySum`. You'll want to use the `sum()` function.

```
aList <- list(c(1, 5, 4), letters[c(1, 6, 4, 9, 22, 3)], list(c(1, 1, 1),
                                                             c(14, 13, 12), c(3, 2, 1)), c(runif(8)))

mySum <- sum(aList[[3]][[2]])
mySum
```

```
## [1] 39
```

(5.2) What is the difference between what is returned from the following two commands?

```
aList[[3]][2]
```

```
## [[1]]
## [1] 14 13 12
```

```
aList[[3]][[2]]
```

For `aList[[3]][2]` (the first one), it returns the second item from the third item of the list AS A LIST. In contrast, `alist[[3]][[2]]` (the second one) returns the second item from the third item of the list AS A VECTOR.

(6) Pseudo-Coding Your boss at the comes to you to tell you that the mayor wants a report that analyzes various data the city government has collected across multiple departments over the last 35 years. This data was created by an army of summer undergraduate interns, and has a mix of numeric, character, and categorical/factor data. The spreadsheet you are given has one row per neighborhood. For each neighborhood, there are measurements of crime statistics, housing prices, number of parks, number of grocery stores, nubmer of driveways, residence types, tax amount, tax burden, past evictions, and so on. Each row has measurements for one year, so there are 35 rows per neighborhood (and of course, lots of missing data).

Using only the tools you've learned so far, your job is to clean, process, and begin to wrangle this data in R and present some preliminary plots of the data you have observed. Write out in words the steps you would take to better understand the spreadsheet, learn what data it contains, and some graphs you might generate based on the tools used in the homework above. DO NOT INCLUDE ANY R-CODE.

1) Import and Inspect

- `read.csv()` data into `df`
- `names(df)`, `dim(df)`, `str(df)`, `summary(df)`

2) Preprocessing

- Find rows and columns with lots of missing data (can use `complete.cases()`)
- Identify cells with illogical data (negative for number of drivers ways, decimal for number of parks, etc.)
- Change cells with illogical data to empty
- Remove rows and columns that have high amounts of missing data (this will capture the illogical data too)
 - Another option is to impute the data if deemed possible
- Check for repeat rows (i.e., same neighborhood and same year)
 - Drop the row with more missing/illogical data
- Standardize columns
 - Convert each individual column to be the same type. For example, a column may have the same data as numbers, but can be numeric or characters.
 - Convert all columns to some type of number, whether that be `num`, `int`, or `Factor`.

3) Processing

- Line graph for housing prices
 - Housing price on the y-axis
 - Year on the x-axis
 - Each neighborhood gets a unique line with a unique color, all line width 2
 - Include legend in graph
- Scatterplot for tax burden and past evictions
 - Past evictions on the y-axis
 - Tax burden on the x-axis
 - Each year for each neighborhood gets a dot on this scatterplot
 - Include a regression line
 - Each year gets a unique color following a color gradient to visualize time